# MAT 167 — Applied Linear Algebra | Final Project

## Google's PageRank Algorithm

**UC Davis · Winter Quarter 2026 · Instructor: DP Varn**
**Network:** Google Network #10 · **Keywords:** Trees Set #3
**Authors:** Arezoo Ghasemzadeh, Ankita Khatri, Sanjay Manivasagam

| Section | Content | Project Step(s) |
|---|---|---|
| 1 | Imports & Setup | — |
| 2 | Define Network & Build Raw Google Matrix G | Step 1 |
| 3 | Eigendecomposition of Raw G + Diagnostics | Steps 2–3 |
| 4 | Fix Dangling Nodes | Step 4 |
| 5 | Eigendecomposition of Fixed G | Step 5 |
| 6 | Normalize π and Rank Pages | Steps 6–7 |
| 7 | Power Method | Step 8 |
| 8 | Teleportation Matrix $\tilde{G}$ | Step 9 |
| 9 | Eigendecomposition of $\tilde{G}$ | Step 10 |
| 10 | Compare Rankings: G_fixed vs $\tilde{G}$ | Step 11 |
| 11 | Term-Document Matrix T | Step 12 |
| 12 | Search Queries (SINGLE, AND, OR, BUT NOT) | Steps 13–15 |
| 13 | Complete Results Summary | — |

## Section 1 — Imports & Setup

Three libraries needed:

- **numpy**: all matrix/vector operations
- **scipy.linalg.eig**: full eigendecomposition (more stable than `numpy.linalg.eig` for stochastic matrices)
- **matplotlib**: bar charts, convergence plots, heatmaps

In [1]:
```
import numpy as np
from scipy.linalg import eig
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
```

```
plt.rcParams.update({
    'figure.figsize': (10, 5),
    'font.size':      12,
    'axes.titlesize': 14,
    'axes.labelsize': 12,
})

print('All imports successful.')
print(f'NumPy version: {np.__version__}')
```

```
All imports successful.
NumPy version: 1.26.4
```

# Section 2 — Define Network & Build Raw Google Matrix G

**(Project Instruction Step 1 | Saito Lecture 25, Slide 6)**

G_{ij} = \\begin{cases} \\frac{1}{|P_i|} & \\text{if page } i \\text{ links to page } j \\\\ 0 & \\t

Each outlink gets equal weight. Every row must sum to **1.0** (normal page) or **0.0** (dangling node).
A row-stochastic matrix is the transition matrix of the random-surfer Markov chain.

**Node index convention:** A=0, B=1, C=2, D=3, E=4, F=5, G=6, H=7, I=8, J=9, K=10, L=11

In [2]:
```
n = 12
labels = list('ABCDEFGHIJKL')   # labels[0]='A', ..., labels[11]='L'

page_titles = {
    'A': 'Trees For All',      'B': 'The Weeping Willow',
    'C': 'Leaf Fall',          'D': 'Shady Place',
    'E': 'The Prickly Pine',   'F': 'Good Wood',
    'G': 'Lumber Jack Man',    'H': 'Timber!',
    'I': 'Fine Grains',        'J': 'The Warped Board',
    'K': 'Evergreen Plants',   'L': 'Sappy Planks',
}

# Adjacency list: adj[i] = list of node indices that page i links TO (outlin
# Source: GoogleNet10 hand-drawn diagram. VERIFY against diagram before runn
adj = {
    0:  [1, 5],          # A -> B, F
    1:  [0, 2, 5, 6],    # B -> A, C, F, G
    2:  [3, 6],          # C -> D, G
    3:  [0, 7],          # D -> A, H
    4:  [0, 5, 8],       # E -> A, F, I
    5:  [0, 8, 9],       # F -> A, I, J
    6:  [3, 5, 7, 10],   # G -> D, F, H, K
    7:  [11],            # H -> L
    8:  [4, 5, 9],       # I -> E, F, J
    9:  [6],             # J -> G
    10: [9],             # K -> J
```

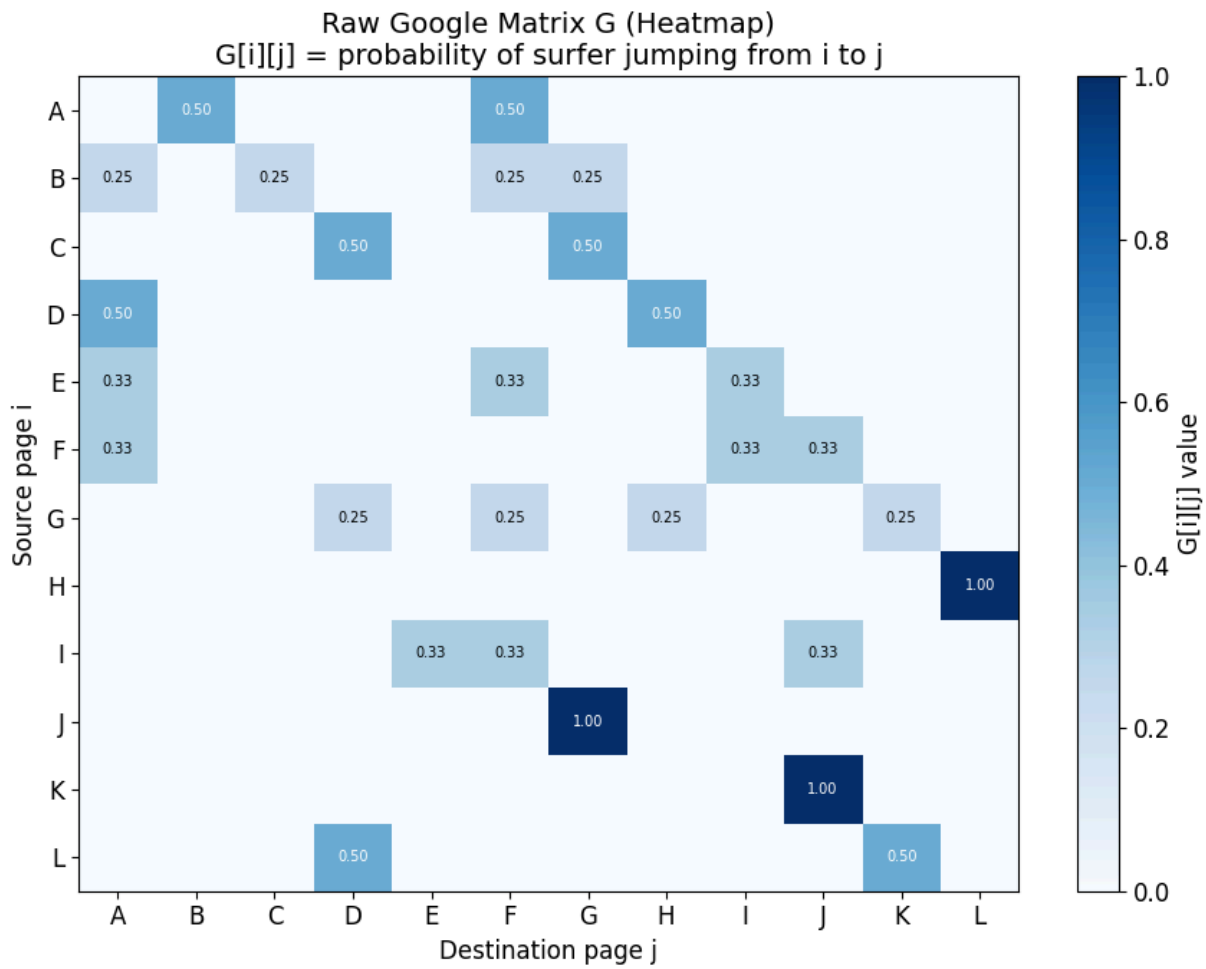
```
    11: [3, 10],         # L -> D, K
}

def build_google_matrix(adj, n):
    """
    Build the raw Google matrix G from an adjacency list.
    G[i][j] = 1/len(outlinks_from_i)  if i links to j, else 0.
    Rows with no outlinks stay all-zero (dangling nodes).
    """
    G = np.zeros((n, n))
    for i in range(n):
        outlinks = adj.get(i, [])
        if len(outlinks) > 0:
            for j in outlinks:
                G[i][j] = 1.0 / len(outlinks)
    return G

G_raw = build_google_matrix(adj, n)

# Validation
row_sums = G_raw.sum(axis=1)
print('Row sums of G_raw (must be 0.0 or 1.0):')
for i, s in enumerate(row_sums):
    status = 'OK' if abs(s - 1.0) < 1e-10 or abs(s) < 1e-10 else 'ERROR'
    print(f'  Page {labels[i]}: {s:.6f}  {status}')

dangling = [labels[i] for i, s in enumerate(row_sums) if abs(s) < 1e-10]
print(f'\nDangling nodes (zero row sum): {dangling if dangling else "None"}'
```

```
Row sums of G_raw (must be 0.0 or 1.0):
  Page A: 1.000000  OK
  Page B: 1.000000  OK
  Page C: 1.000000  OK
  Page D: 1.000000  OK
  Page E: 1.000000  OK
  Page F: 1.000000  OK
  Page G: 1.000000  OK
  Page H: 1.000000  OK
  Page I: 1.000000  OK
  Page J: 1.000000  OK
  Page K: 1.000000  OK
  Page L: 1.000000  OK

Dangling nodes (zero row sum): None
```

In [3]:

```
# Heatmap of G_raw  (Slide 13)
fig, ax = plt.subplots(figsize=(9, 7))
im = ax.imshow(G_raw, cmap='Blues', aspect='auto', vmin=0)
plt.colorbar(im, ax=ax, label='G[i][j] value')
ax.set_xticks(range(n)); ax.set_xticklabels(labels)
ax.set_yticks(range(n)); ax.set_yticklabels(labels)
ax.set_xlabel('Destination page j')
ax.set_ylabel('Source page i')
ax.set_title('Raw Google Matrix G (Heatmap)\nG[i][j] = probability of surfer
for i in range(n):
    for j in range(n):
```

```
        if G_raw[i, j] > 0:
            ax.text(j, i, f'{G_raw[i,j]:.2f}', ha='center', va='center',
                    fontsize=7, color='white' if G_raw[i,j] > 0.4 else 'blac
plt.tight_layout()
plt.savefig('G_raw_heatmap.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved: G_raw_heatmap.png')
```




```
Saved: G_raw_heatmap.png
```

# Section 3 — Eigendecomposition of Raw G + Diagnostics

**(Project Instruction Steps 2–3 | Saito Lecture 25, Slides 6–7)**

The PageRank equation:

$pi^T G =$

$pi^T$

$iffG^T$

$pi =$

$pi$

$pi$

is the **right eigenvector of** $G^T$ for eigenvalue
$lambda = 1$
.

**Key diagnostic:** If $G$ has dangling nodes, it is not row-stochastic, so
$lambda_{max} < 1$
.
The dominant eigenvector may then have **mixed signs** and cannot represent a probability distribution.
For Network #10 there are no dangling nodes, but we run this check to demonstrate understanding.

In [4]:
```
def compute_pagerank_eig(G, label='G'):
    """
    Compute PageRank via direct eigendecomposition of G^T.
    Returns: (pi, dominant_eigenvalue, all_eigenvalues)
    """
    eigenvalues, eigenvectors = eig(G.T)
    idx        = np.argmax(np.real(eigenvalues))
    lambda_max = np.real(eigenvalues[idx])
    pi = np.real(eigenvectors[:, idx])
    pi = np.abs(pi)       # arbitrary sign -> force non-negative
    pi = pi / pi.sum()    # L1-normalize -> valid probability distribution
    print(f"\n{'='*50}")
    print(f'Eigendecomposition of {label}')
    print(f"{'='*50}")
    print(f'  Dominant eigenvalue (lambda_max): {lambda_max:.10f}')
    print(f'  All eigenvalues (real parts): {sorted(np.real(eigenvalues), re
    print(f'  sum(pi) after normalization:  {pi.sum():.10f}  (must be 1.0)')
    print(f'  min(pi):                      {pi.min():.10f}  (must be >= 0)'
    return pi, lambda_max, eigenvalues

# Step 2: eigendecomposition of raw G
pi_raw, lambda_raw, evals_raw = compute_pagerank_eig(G_raw, 'G_raw')

# Mixed-sign diagnostic
dominant_idx_raw = np.argmax(np.real(evals_raw))
raw_vec = np.real(eig(G_raw.T)[1][:, dominant_idx_raw])
has_mixed_signs = not (np.all(raw_vec >= -1e-12) or np.all(raw_vec <= 1e-12)

print('\nMixed-sign diagnostic on raw G dominant eigenvector:')
if has_mixed_signs:
    print('  WARNING: eigenvector has MIXED SIGNS — cannot represent a proba
    print('  Fix: replace dangling-node rows (Section 4).')
else:
    print('  OK: eigenvector is non-negative.')
    print('  (Network #10: no dangling nodes, raw G is already valid.)')

print(f'\nPageRank vector from raw G (before any fixes):')
print(f"  {'Page':<6} {'Title':<25} {'pi value'}")
print(f"  {'-'*44}")
for i in range(n):
```

```
    print(f'  {labels[i]:<6} {page_titles[labels[i]]:<25} {pi_raw[i]:.6f}')
print(f'\n  lambda_max = {lambda_raw:.4f}  (expected 1.0 for a fully row-stc
```

```
==================================================
Eigendecomposition of G_raw
==================================================
  Dominant eigenvalue (lambda_max): 1.0000000000
  All eigenvalues (real parts): [1.0000000000000018, 0.625273785016588, 0.39
66543094646142, 0.0470618727101512, 0.0470618727101512, -0.0538875835029308
8, -0.05388758350293088, -0.21774244358192865, -0.21774244358192865, -0.4794
9354397940336, -0.47949354397940336, -0.6138046977729764]
  sum(pi) after normalization:  1.0000000000  (must be 1.0)
  min(pi):                      0.0131201930  (must be >= 0)

Mixed-sign diagnostic on raw G dominant eigenvector:
  OK: eigenvector is non-negative.
  (Network #10: no dangling nodes, raw G is already valid.)

PageRank vector from raw G (before any fixes):
  Page   Title                     pi value
  -------------------------------------------
  A      Trees For All             0.104962
  B      The Weeping Willow        0.052481
  C      Leaf Fall                 0.013120
  D      Shady Place               0.088674
  E      The Prickly Pine          0.015835
  F      Good Wood                 0.126678
  G      Lumber Jack Man           0.159855
  H      Timber!                   0.084301
  I      Fine Grains               0.047504
  J      The Warped Board          0.140175
  K      Evergreen Plants          0.082114
  L      Sappy Planks              0.084301

  lambda_max = 1.0000  (expected 1.0 for a fully row-stochastic G)
```

# Section 4 — Fix Dangling Nodes → Make G Row-Stochastic

**(Project Instruction Step 4 | Saito Lecture 25, Slide 9)**

A dangling node has no outlinks → its row in $G$ is all zeros.

**Fix:** replace each all-zero row with

$[1/n, 1/n,$

$ldots, 1/n]$

.

This models a surfer teleporting to a random page when stuck at a dead end.

**Network #10 result:** No dangling nodes found. All 12 rows already sum to 1.0.

In [5]:
```
G_fixed = G_raw.copy()
fixed_list = []
```

```
for i in range(n):
    if G_fixed[i].sum() < 1e-10:
        G_fixed[i] = 1.0 / n
        fixed_list.append(labels[i])

if fixed_list:
    print(f'Fixed dangling nodes: {fixed_list}')
else:
    print('No dangling nodes found -- G_raw is already fully row-stochastic.
    print('(Network #10 result: all 12 pages have at least one outlink.)')

row_sums_fixed = G_fixed.sum(axis=1)
all_ok = all(abs(s - 1.0) < 1e-10 for s in row_sums_fixed)
print(f'\nAll rows sum to 1? {"Yes" if all_ok else "No -- check adjacency li
```

```
No dangling nodes found -- G_raw is already fully row-stochastic.
(Network #10 result: all 12 pages have at least one outlink.)

All rows sum to 1? Yes
```

# Section 5 — Eigendecomposition of Fixed G

**(Project Instruction Step 5 | Saito Lecture 25, Slide 10)**

Now $G$ is fully row-stochastic. The **Perron-Frobenius Theorem** guarantees:

1. $lambda_{max} = 1$
   exactly
2. The corresponding eigenvector has **all non-negative entries**
3. The eigenvector is unique (up to scaling) if the Markov chain is irreducible

In [6]:
```
pi_fixed, lambda_fixed, evals_fixed = compute_pagerank_eig(G_fixed, 'G_fixed
print(f'\n  lambda_max = {lambda_fixed:.10f}  (should be 1.0)')
print(f'  All entries non-negative: {(pi_fixed >= -1e-12).all()}')
```

```
==================================================
Eigendecomposition of G_fixed
==================================================
  Dominant eigenvalue (lambda_max): 1.0000000000
  All eigenvalues (real parts): [1.0000000000000018, 0.625273785016588, 0.39
66543094646142, 0.0470618727101512, 0.0470618727101512, -0.0538875835029308
8, -0.05388758350293088, -0.21774244358192865, -0.21774244358192865, -0.4794
9354397940336, -0.47949354397940336, -0.6138046977729764]
  sum(pi) after normalization:  1.0000000000  (must be 1.0)
  min(pi):                      0.0131201930  (must be >= 0)

  lambda_max = 1.0000000000  (should be 1.0)
  All entries non-negative: True
```

# Section 6–7 — Normalize π and Rank Pages

**(Project Instruction Steps 6–7)**

**Step 6:**
$pi$
is already L1-normalized inside `compute_pagerank_eig()`:
$|$
$pi$
$|_1 =$
$sum_i$
$pi_i = 1$

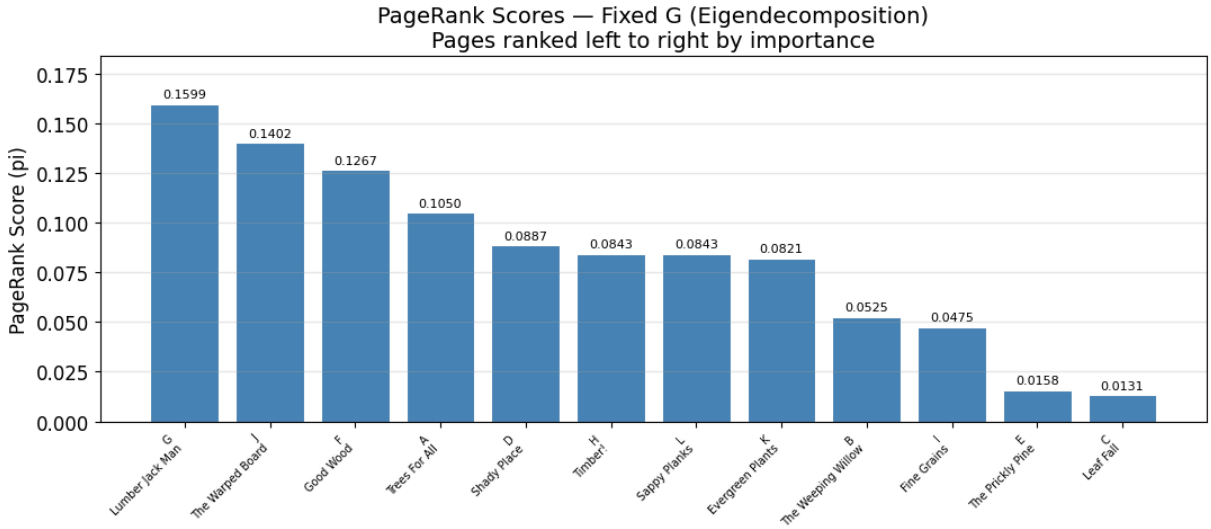
**Step 7:** Sort entries of
$pi$
in **non-increasing order**. `np.argsort(-pi)` gives descending indices.

In [7]:
```
def rank_pages(pi, labels, page_titles, header='PageRank Ranking'):
    """Sort pages by PageRank descending and print a formatted table."""
    sorted_idx = np.argsort(-pi)
    print(f'\n{header}')
    print(f"  {'Rank':<6} {'Page':<6} {'Title':<25} {'pi value':<12} {'% of
    print('  ' + '-' * 65)
    for rank, i in enumerate(sorted_idx, start=1):
        pct = pi[i] * 100
        bar = '#' * int(pct * 2)
        print(f'  #{rank:<5} {labels[i]:<6} {page_titles[labels[i]]:<25} '
              f'{pi[i]:.6f}    {pct:.2f}% {bar}')
    print(f'\n  sum(pi) = {pi.sum():.10f}  (must be 1.0)')
    return sorted_idx

print(f'Verification: sum(pi_fixed) = {pi_fixed.sum():.12f}')
sorted_fixed = rank_pages(pi_fixed, labels, page_titles,
                          'PageRank from G_fixed (Eigendecomposition)')
```

```
Verification: sum(pi_fixed) = 1.000000000000

PageRank from G_fixed (Eigendecomposition)
  Rank   Page   Title                     pi value     % of total
  ------------------------------------------------------------------
  #1     G      Lumber Jack Man           0.159855    15.99% ###############
################
  #2     J      The Warped Board          0.140175    14.02% ###############
#############
  #3     F      Good Wood                 0.126678    12.67% ###############
##########
  #4     A      Trees For All             0.104962    10.50% ###############
#####
  #5     D      Shady Place               0.088674    8.87% ################
#
  #6     H      Timber!                   0.084301    8.43% ################
  #7     L      Sappy Planks              0.084301    8.43% ################
  #8     K      Evergreen Plants          0.082114    8.21% ################
  #9     B      The Weeping Willow        0.052481    5.25% ##########
  #10    I      Fine Grains               0.047504    4.75% #########
  #11    E      The Prickly Pine          0.015835    1.58% ###
  #12    C      Leaf Fall                 0.013120    1.31% ##

  sum(pi) = 1.0000000000  (must be 1.0)
```

In [8]:

```python
# Bar chart (Slide 14)
fig, ax = plt.subplots(figsize=(11, 5))
idx_s = np.argsort(-pi_fixed)
xlbls = [f'{labels[i]}\n{page_titles[labels[i]]}' for i in idx_s]
pi_s  = pi_fixed[idx_s]
bars  = ax.bar(range(n), pi_s, color='steelblue', edgecolor='white', linewic
for bar, val in zip(bars, pi_s):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.002,
            f'{val:.4f}', ha='center', va='bottom', fontsize=8)
ax.set_xticks(range(n)); ax.set_xticklabels(xlbls, fontsize=7, rotation=45,
ax.set_ylabel('PageRank Score (pi)')
ax.set_title('PageRank Scores — Fixed G (Eigendecomposition)\nPages ranked l
ax.set_ylim(0, max(pi_s) * 1.15)
ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig('pagerank_fixed_G.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved: pagerank_fixed_G.png')
```




```
Saved: pagerank_fixed_G.png
```

# Section 7 — Power Method

**(Project Instruction Step 8 | Saito Lecture 24 + Lecture 25, Slide 6)**

Direct eigendecomposition requires $O(n^3)$ — infeasible for Google's
$n$
$approx3$
$times10^8$
pages.

The Power Method only computes matrix-vector products and exploits sparsity.

**Update rule:**
$pi_j^T =$
$pi_{j-1}^T$
$cdotG$
, then normalize

**Start:**
$pi_0 = [1/n,$
$ldots, 1/n]^T$
(uniform)

**Stop:**
$|$
$pi_j -$
$pi_{j-1}$
$|_1$
$leq$
$varepsilon$

**Why it converges:** Any

$pi_0 = c_1 v_1 + c_2 v_2 +$
$cdots$
. After $k$ steps:
$pi_k = c_1$
$lambda_1^k v_1 + c_2$
$lambda_2^k v_2 +$
$cdots$

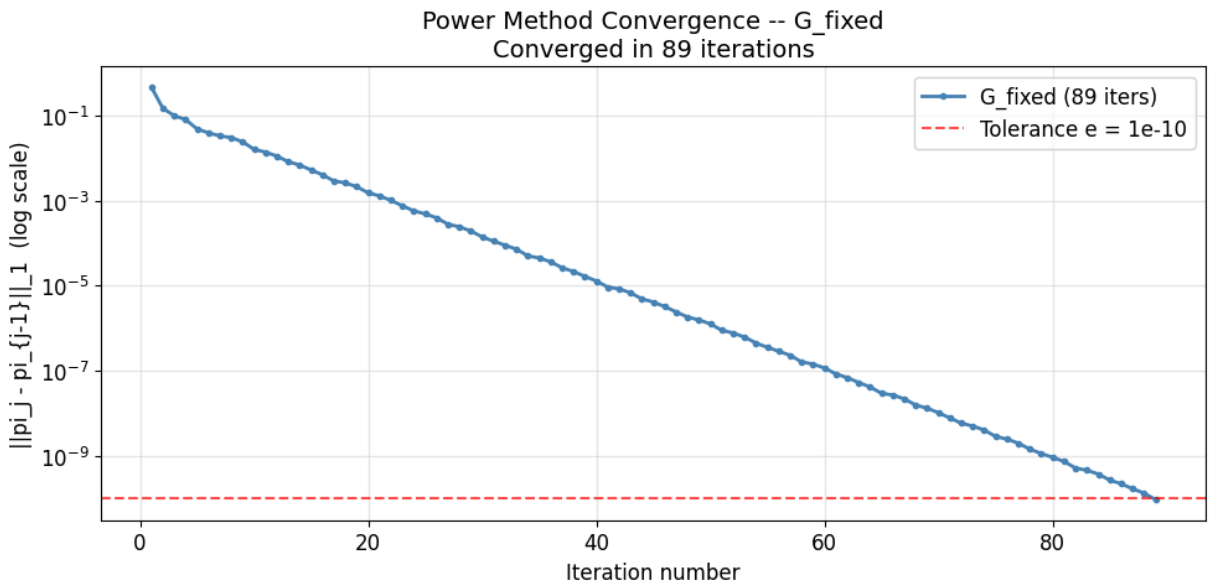
Since
$|$
$lambda_i| < 1$
for
$i$
$geq 2$
, those terms decay. Only the dominant eigenvector $v_1$ survives.

**Critical notation:** `pi @ G` (row vector × G on right) = correct. `G @ pi` = wrong.

In [9]:
```
def power_method(G, tol=1e-10, max_iter=1000, label='G'):
    """
    Find the dominant LEFT eigenvector of G using power iteration.
    Update: pi_new = pi @ G, then normalize.
    Returns: (pi, iteration_count, convergence_history)
    """
    n_loc   = G.shape[0]
    pi      = np.ones(n_loc) / n_loc
    history = []
    for k in range(1, max_iter + 1):
        pi_new = pi @ G
        pi_new = pi_new / pi_new.sum()
        delta  = np.linalg.norm(pi_new - pi, ord=1)
        history.append(delta)
        if delta <= tol:
            print(f'  Power Method [{label}]: converged in {k} iterations (f
            return pi_new, k, history
        pi = pi_new
    print(f'  Power Method [{label}]: WARNING -- did not converge in {max_it
    return pi, max_iter, history

pi_power_fixed, iters_fixed, hist_fixed = power_method(G_fixed, label='G_fix

max_diff_fixed = np.max(np.abs(pi_power_fixed - pi_fixed))
print(f'\n  Max |pi_power - pi_eig|_inf = {max_diff_fixed:.2e}  (should be <
print(f'  Methods agree: {max_diff_fixed < 1e-8}')
```

```
  Power Method [G_fixed]: converged in 89 iterations (final delta = 9.58e-1
1)

  Max |pi_power - pi_eig|_inf = 1.37e-11  (should be < 1e-8)
  Methods agree: True
```

In [10]:
```
# Convergence plot (Slide 15)
fig, ax = plt.subplots(figsize=(10, 5))
ax.semilogy(range(1, len(hist_fixed) + 1), hist_fixed,
            color='steelblue', linewidth=2, marker='o', markersize=3,
            label=f'G_fixed ({iters_fixed} iters)')
ax.axhline(y=1e-10, color='red', linestyle='--', alpha=0.7, label='Tolerance
ax.set_xlabel('Iteration number')
ax.set_ylabel('||pi_j - pi_{j-1}||_1  (log scale)')
ax.set_title(f'Power Method Convergence -- G_fixed\nConverged in {iters_fixe
ax.legend(); ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('convergence_G_fixed.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved: convergence_G_fixed.png')
```




```
Saved: convergence_G_fixed.png
```

## Section 8 — Build the Teleportation Matrix $\tilde{G}$

**(Project Instruction Step 9 | Saito Lecture 25, Slides 14–15)**

Even after fixing dangling nodes, the graph may not be **strongly connected** (reducible Markov chain → non-unique stationary distribution).

**Brin & Page's fix — teleportation:**

$$tildeG = alphaG + (1 - alpha)E, quadE = frac1n mathbf1 mathbf1^T, quad alpha = 0.85$$

- With probability $alpha = 0.85$
  : surfer follows a link normally
- With probability $1 - alpha = 0.15$
  : surfer teleports to any random page

Since every entry of $E = 1/n > 0$, every entry of $tildeG > 0$
→ **irreducible + primitive**.
Perron-Frobenius guarantees a **unique, fully positive** PageRank vector.

In [11]:
```
alpha   = 0.85
E       = np.ones((n, n)) / n
G_tilde = alpha * G_fixed + (1 - alpha) * E

print(f'alpha = {alpha},  (1-alpha) = {1-alpha}')
print(f'E: every entry = 1/{n} = {1/n:.6f}')

row_sums_tilde = G_tilde.sum(axis=1)
all_stoch = all(abs(s - 1.0) < 1e-10 for s in row_sums_tilde)
print(f'\nRow sums:  min={row_sums_tilde.min():.10f}  max={row_sums_tilde.ma
print(f'All rows sum to 1? {"Yes" if all_stoch else "No"}')
print(f'Min entry of G_tilde: {G_tilde.min():.6f}  (must be > 0 for irreduci
print(f'G_tilde is irreducible: {"Yes" if G_tilde.min() > 0 else "No"}')
print('-> Perron-Frobenius guarantees a unique, fully positive PageRank vect
```

```
alpha = 0.85,  (1-alpha) = 0.15000000000000002
E: every entry = 1/12 = 0.083333

Row sums:  min=1.0000000000  max=1.0000000000
All rows sum to 1? Yes
Min entry of G_tilde: 0.012500  (must be > 0 for irreducibility)
G_tilde is irreducible: Yes
-> Perron-Frobenius guarantees a unique, fully positive PageRank vector.
```

## Section 9 — Eigendecomposition of $\tilde{G}$ (the True Google Matrix)

**(Project Instruction Step 10)**

This is the **final, authoritative PageRank** used for all search queries.

$\tilde{G}$ is irreducible and primitive, so all entries of
$pi$
are **strictly > 0**.

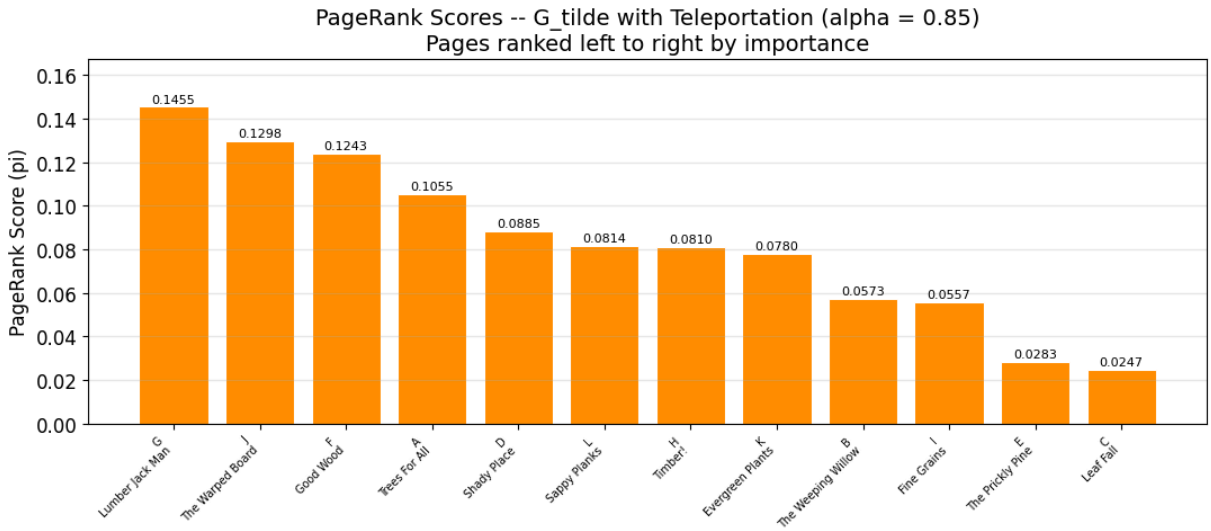
In [12]:
```
pi_tilde, lambda_tilde, evals_tilde = compute_pagerank_eig(G_tilde, 'G_tilde

print(f'\n  lambda_max = {lambda_tilde:.10f}  (should be 1.0)')
print(f'  All entries > 0: {(pi_tilde > 0).all()}  (guaranteed by irreducibi

sorted_tilde = rank_pages(pi_tilde, labels, page_titles,
                          'PageRank from G_tilde (Eigendecomposition)')

# Power method on G_tilde -- verify agreement
pi_power_tilde, iters_tilde, hist_tilde = power_method(G_tilde, label='G_til
max_diff_tilde = np.max(np.abs(pi_power_tilde - pi_tilde))
print(f'\n  Max |pi_power - pi_eig|_inf for G_tilde = {max_diff_tilde:.2e}')
print(f'  Methods agree: {max_diff_tilde < 1e-8}')
```

```
==================================================
Eigendecomposition of G_tilde
==================================================
  Dominant eigenvalue (lambda_max): 1.0000000000
  All eigenvalues (real parts): [1.0000000000000007, 0.5314827172640986, 0.3
371561630449218, 0.04000259180362876, 0.04000259180362876, -0.04580444597749
145, -0.04580444597749145, -0.1850810770446392, -0.1850810770446392, -0.4075
695123824933, -0.4075695123824933, -0.5217339931070326]
  sum(pi) after normalization:  1.0000000000  (must be 1.0)
  min(pi):                      0.0246855561  (must be >= 0)

  lambda_max = 1.0000000000  (should be 1.0)
  All entries > 0: True  (guaranteed by irreducibility)

PageRank from G_tilde (Eigendecomposition)
  Rank   Page   Title                     pi value     % of total
  ------------------------------------------------------------------
  #1     G      Lumber Jack Man           0.145502    14.55% ###############
###############
  #2     J      The Warped Board          0.129794    12.98% ###############
##########
  #3     F      Good Wood                 0.124250    12.43% ###############
#########
  #4     A      Trees For All             0.105515    10.55% ###############
######
  #5     D      Shady Place               0.088495    8.85% ################
#
  #6     L      Sappy Planks              0.081375    8.14% ################
  #7     H      Timber!                   0.081030    8.10% ################
  #8     K      Evergreen Plants          0.078004    7.80% ###############
  #9     B      The Weeping Willow        0.057344    5.73% ###########
  #10    I      Fine Grains               0.055719    5.57% ###########
  #11    E      The Prickly Pine          0.028287    2.83% #####
  #12    C      Leaf Fall                 0.024686    2.47% ####

  sum(pi) = 1.0000000000  (must be 1.0)
  Power Method [G_tilde]: converged in 54 iterations (final delta = 6.95e-1
1)

  Max |pi_power - pi_eig|_inf for G_tilde = 5.98e-12
  Methods agree: True
```

In [13]:

```
# Bar chart for G_tilde (Slide 16)
fig, ax = plt.subplots(figsize=(11, 5))
idx_t           = np.argsort(-pi_tilde)
xlbls_t         = [f'{labels[i]}\n{page_titles[labels[i]]}' for i in idx_t]
pi_tilde_sorted = pi_tilde[idx_t]
bars = ax.bar(range(n), pi_tilde_sorted, color='darkorange', edgecolor='whit
for bar, val in zip(bars, pi_tilde_sorted):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.001,
            f'{val:.4f}', ha='center', va='bottom', fontsize=8)
ax.set_xticks(range(n)); ax.set_xticklabels(xlbls_t, fontsize=7, rotation=45
ax.set_ylabel('PageRank Score (pi)')
ax.set_title('PageRank Scores -- G_tilde with Teleportation (alpha = 0.85)\r
ax.set_ylim(0, max(pi_tilde_sorted) * 1.15)
ax.grid(axis='y', alpha=0.3)
```

```
plt.tight_layout()
plt.savefig('pagerank_G_tilde.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved: pagerank_G_tilde.png')
```




```
Saved: pagerank_G_tilde.png
```

# Section 10 — Compare Rankings: G_fixed vs G̃

**(Project Instruction Step 11)**

Teleportation slightly redistributes importance:

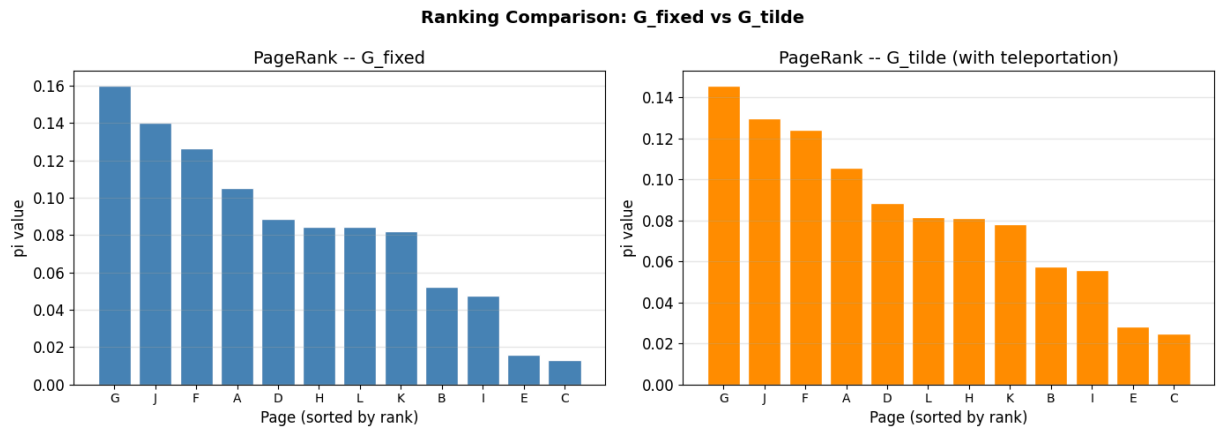
- Top pages **lose a little weight** (15% of visits now go randomly elsewhere)
- Bottom pages **gain a little** (15% of visits now come from everywhere)
- The overall structural ranking is **robust** — link structure dominates

In [14]:
```
rank_fixed_arr = np.argsort(-pi_fixed)
rank_tilde_arr = np.argsort(-pi_tilde)

print('RANKING COMPARISON: G_fixed vs G_tilde')
print(f"  {'Rank':<6} {'G_fixed page':<14} {'pi(G_fixed)':<14} {'G_tilde pag
print('  ' + '-' * 72)
for r in range(n):
    i    = rank_fixed_arr[r]
    j    = rank_tilde_arr[r]
    same = 'Yes' if i == j else '<--changed'
    print(f'  #{r+1:<4} {labels[i]:<14} {pi_fixed[i]:<14.6f} {labels[j]:<14}
```

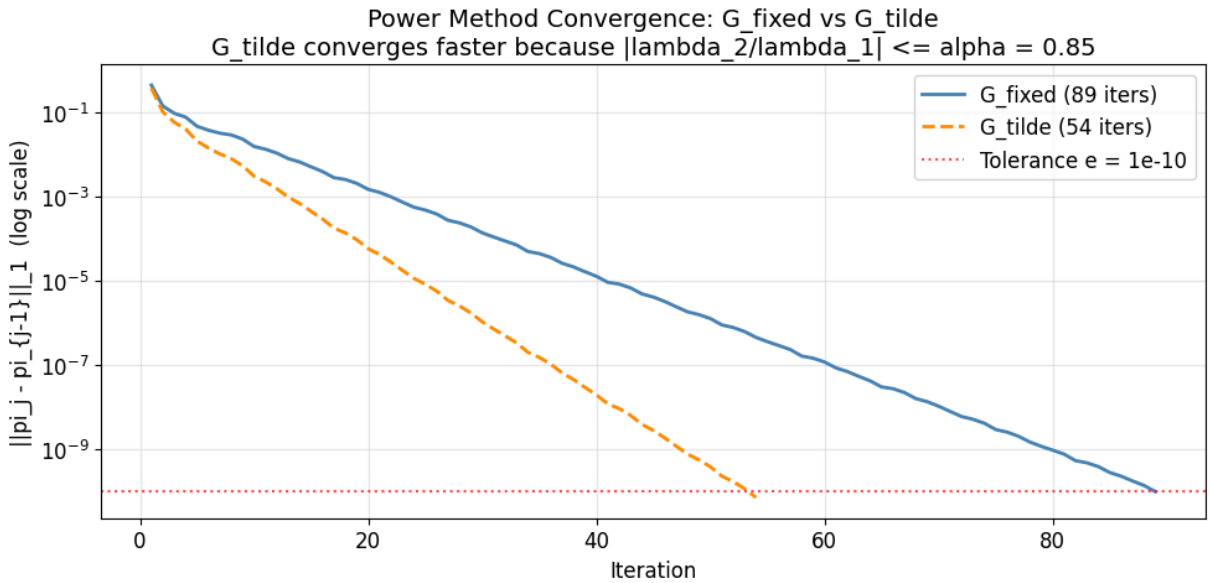
```
RANKING COMPARISON: G_fixed vs G_tilde
  Rank   G_fixed page   pi(G_fixed)    G_tilde page   pi(G_tilde)    Same?
  ---------------------------------------------------------------------------
  #1    G              0.159855       G              0.145502       Yes
  #2    J              0.140175       J              0.129794       Yes
  #3    F              0.126678       F              0.124250       Yes
  #4    A              0.104962       A              0.105515       Yes
  #5    D              0.088674       D              0.088495       Yes
  #6    H              0.084301       L              0.081375       <--chang
ed
  #7    L              0.084301       H              0.081030       <--chang
ed
  #8    K              0.082114       K              0.078004       Yes
  #9    B              0.052481       B              0.057344       Yes
  #10   I              0.047504       I              0.055719       Yes
  #11   E              0.015835       E              0.028287       Yes
  #12   C              0.013120       C              0.024686       Yes
```

In [15]:

```
# Side-by-side bar chart (Slide 17)
fig, axes = plt.subplots(1, 2, figsize=(14, 5), sharey=False)
for ax, pi_vals, color, title in [
    (axes[0], pi_fixed,  'steelblue',  'G_fixed'),
    (axes[1], pi_tilde,  'darkorange', 'G_tilde (with teleportation)'),
]:
    idx_c = np.argsort(-pi_vals)
    ax.bar(range(n), pi_vals[idx_c], color=color, edgecolor='white')
    ax.set_xticks(range(n))
    ax.set_xticklabels([labels[i] for i in idx_c], fontsize=10)
    ax.set_title(f'PageRank -- {title}')
    ax.set_ylabel('pi value')
    ax.set_xlabel('Page (sorted by rank)')
    ax.grid(axis='y', alpha=0.3)
plt.suptitle('Ranking Comparison: G_fixed vs G_tilde', fontsize=14, fontweig
plt.tight_layout()
plt.savefig('ranking_comparison.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved: ranking_comparison.png')

# Combined convergence plot (Slide 15)
fig, ax = plt.subplots(figsize=(10, 5))
ax.semilogy(range(1, len(hist_fixed) + 1), hist_fixed,
            color='steelblue', linewidth=2, label=f'G_fixed ({iters_fixed} i
ax.semilogy(range(1, len(hist_tilde) + 1), hist_tilde,
            color='darkorange', linewidth=2, linestyle='--',
            label=f'G_tilde ({iters_tilde} iters)')
ax.axhline(y=1e-10, color='red', linestyle=':', alpha=0.7, label='Tolerance
ax.set_xlabel('Iteration')
ax.set_ylabel('||pi_j - pi_{j-1}||_1  (log scale)')
ax.set_title(f'Power Method Convergence: G_fixed vs G_tilde\nG_tilde converg
ax.legend(); ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('convergence_both.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved: convergence_both.png')
```




```
Saved: ranking_comparison.png
```




```
Saved: convergence_both.png
```

# Section 11 — Term-Document Matrix T

**(Project Instruction Step 12 | Saito Lecture 25, Slide 11)**

$T$ is a **17 × 12** binary matrix (17 keywords × 12 pages):

T_{kp} = \\begin{cases} 1 & \\text{if keyword } k \\text{ appears on page } p \\\\ 0 & \\tex

For a query vector $q$ (length 17), the product $d = T^T q$ gives $d_p$ = number of query keywords on page $p$.

**Source:** TermDoc03 PDF — *Final Project WebPage KeyWords #3: Trees*

In [16]:
```
keywords = [
    'Ash','Aspen','Butternut','Cherry','Elm','Fir','Ginkgo','Hickory',
    'Katsura','Magnolia','Oak','Pine','Redwood','Sassafras','Spruce','Teak',
]
kw = {k: i for i, k in enumerate(keywords)}

# Pages:   A  B  C  D  E  F  G  H  I  J  K  L
```

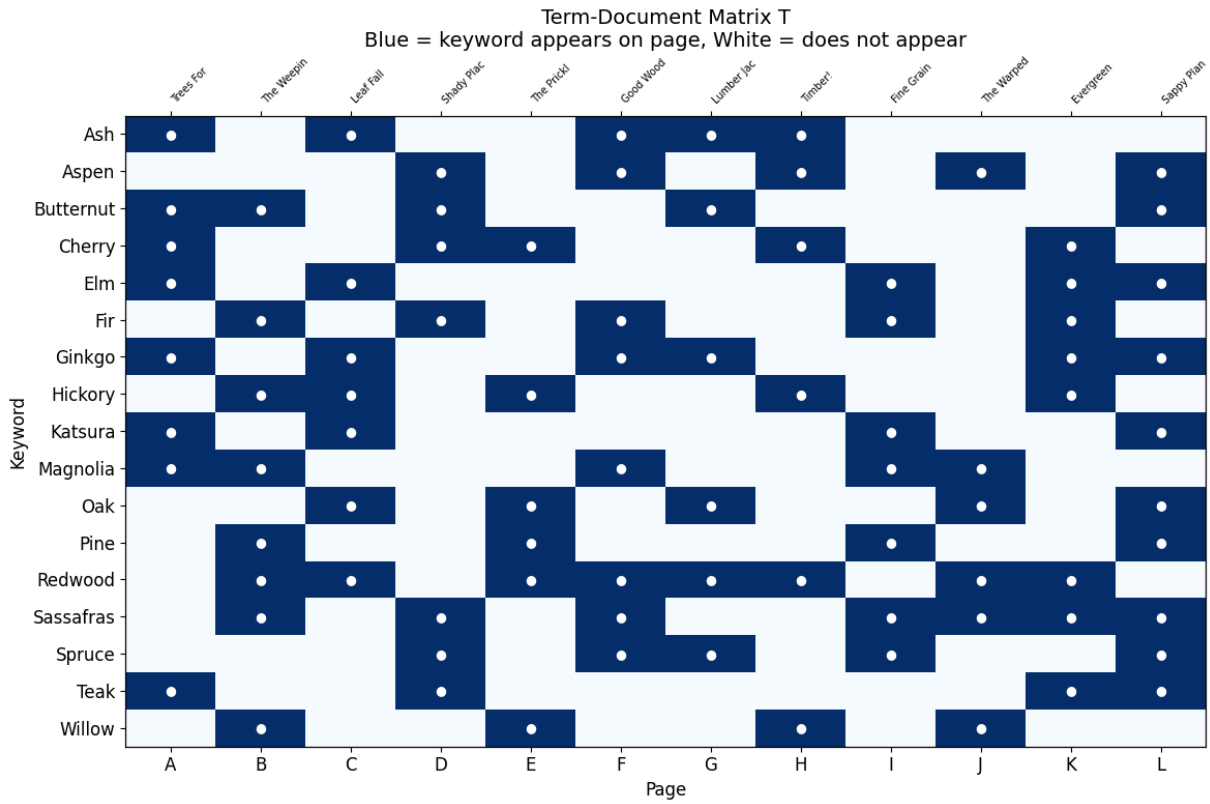
```
T = np.array([
  [1, 0, 1, 0, 0, 1, 1, 1, 0, 0, 0, 0],  # Ash
  [0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1],  # Aspen
  [1, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1],  # Butternut
  [1, 0, 0, 1, 1, 0, 0, 1, 0, 0, 1, 0],  # Cherry
  [1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 1, 1],  # Elm
  [0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0],  # Fir
  [1, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1, 1],  # Ginkgo
  [0, 1, 1, 0, 1, 0, 0, 1, 0, 0, 1, 0],  # Hickory
  [1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1],  # Katsura
  [1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0],  # Magnolia
  [0, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1],  # Oak
  [0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1],  # Pine
  [0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0],  # Redwood
  [0, 1, 0, 1, 0, 1, 0, 0, 1, 1, 1, 1],  # Sassafras
  [0, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1],  # Spruce
  [1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1],  # Teak
  [0, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0],  # Willow
], dtype=float)

print(f'T shape: {T.shape}  (expected (17, 12))')
print('\nKeywords per page:')
for p in range(n):
    kws = [keywords[k] for k in range(len(keywords)) if T[k, p] == 1]
    print(f'  {labels[p]} ({page_titles[labels[p]]}): {", ".join(kws)}')
```

```
T shape: (17, 12)  (expected (17, 12))

Keywords per page:
  A (Trees For All): Ash, Butternut, Cherry, Elm, Ginkgo, Katsura, Magnolia,
Teak
  B (The Weeping Willow): Butternut, Fir, Hickory, Magnolia, Pine, Redwood,
Sassafras, Willow
  C (Leaf Fall): Ash, Elm, Ginkgo, Hickory, Katsura, Oak, Redwood
  D (Shady Place): Aspen, Butternut, Cherry, Fir, Sassafras, Spruce, Teak
  E (The Prickly Pine): Cherry, Hickory, Oak, Pine, Redwood, Willow
  F (Good Wood): Ash, Aspen, Fir, Ginkgo, Magnolia, Redwood, Sassafras, Spru
ce
  G (Lumber Jack Man): Ash, Butternut, Ginkgo, Oak, Redwood, Spruce
  H (Timber!): Ash, Aspen, Cherry, Hickory, Redwood, Willow
  I (Fine Grains): Elm, Fir, Katsura, Magnolia, Pine, Sassafras, Spruce
  J (The Warped Board): Aspen, Magnolia, Oak, Redwood, Sassafras, Willow
  K (Evergreen Plants): Cherry, Elm, Fir, Ginkgo, Hickory, Redwood, Sassafra
s, Teak
  L (Sappy Planks): Aspen, Butternut, Elm, Ginkgo, Katsura, Oak, Pine, Sassa
fras, Spruce, Teak
```

In [17]:

```
# Heatmap of T (Slide 18)
fig, ax = plt.subplots(figsize=(12, 8))
im = ax.imshow(T, cmap='Blues', aspect='auto', vmin=0, vmax=1)
ax.set_xticks(range(n)); ax.set_xticklabels([labels[p] for p in range(n)])
ax.set_yticks(range(len(keywords))); ax.set_yticklabels(keywords)
ax.set_xlabel('Page'); ax.set_ylabel('Keyword')
ax.set_title('Term-Document Matrix T\nBlue = keyword appears on page, White
for k in range(len(keywords)):
    for p in range(n):
```

```
        if T[k, p] == 1:
            ax.text(p, k, '●', ha='center', va='center', color='white', font
ax2 = ax.twiny()
ax2.set_xlim(ax.get_xlim())
ax2.set_xticks(range(n))
ax2.set_xticklabels([page_titles[labels[p]][:10] for p in range(n)],
                    rotation=45, ha='left', fontsize=7)
plt.tight_layout()
plt.savefig('term_document_matrix.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved: term_document_matrix.png')
```




```
Saved: term_document_matrix.png
```

# Section 12 — Search Queries

**(Project Instruction Steps 13–15 | Saito Lecture 25, Slide 11)**

**Process for every query:**

1. Build query vector $q$ (length 17, 1s for searched keywords)
2. Compute $d = T^T q$ — d[p] = number of query keywords on page p
3. Apply Boolean logic → responsive set
4. Sort by G˜PageRank (descending)

| Query type | Responsive if... |
|---|---|
| SINGLE | `d[p] > 0` |
| AND | `d[p] == number of terms` |

| Query type | Responsive if... |
|---|---|
| OR | `d[p] >= 1` |
| BUT NOT | `T[A][p]==1 AND T[B][p]==0` (set operations) |

In [18]:
```
page_keywords = {
    'A': ['Ash','Butternut','Cherry','Elm','Ginkgo','Katsura','Magnolia','Te
    'B': ['Butternut','Fir','Hickory','Magnolia','Pine','Redwood','Sassafras
    'C': ['Ash','Elm','Ginkgo','Hickory','Katsura','Oak','Redwood'],
    'D': ['Aspen','Butternut','Cherry','Fir','Sassafras','Spruce','Teak'],
    'E': ['Cherry','Hickory','Oak','Pine','Redwood','Willow'],
    'F': ['Ash','Aspen','Fir','Ginkgo','Magnolia','Redwood','Sassafras','Spr
    'G': ['Ash','Butternut','Ginkgo','Oak','Redwood','Spruce'],
    'H': ['Ash','Aspen','Cherry','Hickory','Redwood','Willow'],
    'I': ['Elm','Fir','Katsura','Magnolia','Pine','Sassafras','Spruce'],
    'J': ['Aspen','Magnolia','Oak','Redwood','Sassafras','Willow'],
    'K': ['Cherry','Elm','Fir','Ginkgo','Hickory','Redwood','Sassafras','Tea
    'L': ['Aspen','Butternut','Elm','Ginkgo','Katsura','Oak','Pine','Sassafr
}

def show_query_results(query_name, query_keywords_list, responsive_idx,
                       pi_tilde, labels, page_titles, page_keywords):
    """Sort responsive pages by G_tilde PageRank and print with matching key
    ranked = sorted(responsive_idx, key=lambda i: -pi_tilde[i])
    print(f"\n  {'='*60}")
    print(f'  QUERY: {query_name}')
    print(f"  {'='*60}")
    if not ranked:
        print('  -> No pages match this query.')
        return ranked
    print(f"  {'Result':<8} {'Page':<6} {'Title':<25} {'PageRank':<12} {'Mat
    print(f"  {'-'*75}")
    for res_rank, i in enumerate(ranked, start=1):
        matching = [kw for kw in query_keywords_list if kw in page_keywords.
        matching_str = ', '.join(matching)
        print(f'  #{res_rank:<6} {labels[i]:<6} {page_titles[labels[i]]:<25}
              f'{pi_tilde[i]:.6f}   {matching_str}')
    return ranked
```

In [19]:
```
# Query 1: SINGLE -- 'Elm'
q1 = np.zeros(len(keywords)); q1[kw['Elm']] = 1
d1 = T.T @ q1
print('-- Query 1: SINGLE Elm --')
for p in range(n):
    mark = '<-- responsive' if d1[p] > 0 else ''
    print(f'  Page {labels[p]}: d = {d1[p]:.0f}  {mark}')
responsive_1 = [p for p in range(n) if d1[p] > 0]
results_1 = show_query_results('SINGLE: Elm', ['Elm'], responsive_1,
                               pi_tilde, labels, page_titles, page_keywords)
```

```
-- Query 1: SINGLE Elm --
  Page A: d = 1  <-- responsive
  Page B: d = 0
  Page C: d = 1  <-- responsive
  Page D: d = 0
  Page E: d = 0
  Page F: d = 0
  Page G: d = 0
  Page H: d = 0
  Page I: d = 1  <-- responsive
  Page J: d = 0
  Page K: d = 1  <-- responsive
  Page L: d = 1  <-- responsive

  ============================================================
  QUERY: SINGLE: Elm
  ============================================================
  Result   Page   Title                      PageRank     Matching Keywords
  ---------------------------------------------------------------------------
-
  #1      A      Trees For All             0.105515   Elm
  #2      L      Sappy Planks              0.081375   Elm
  #3      K      Evergreen Plants          0.078004   Elm
  #4      I      Fine Grains               0.055719   Elm
  #5      C      Leaf Fall                 0.024686   Elm
```

In [20]:

```python
# Query 2: AND -- 'Aspen AND Sassafras'
q2 = np.zeros(len(keywords))
q2[kw['Aspen']] = 1; q2[kw['Sassafras']] = 1
d2 = T.T @ q2
num_terms_2 = int(q2.sum())
print(f'-- Query 2: AND Aspen AND Sassafras (requires d == {num_terms_2}) --
for p in range(n):
    if d2[p] == num_terms_2: note = '<-- BOTH present (responsive!)'
    elif d2[p] > 0:          note = f'<-- only {int(d2[p])}/{num_terms_2} te
    else:                    note = ''
    print(f'  Page {labels[p]}: d = {d2[p]:.0f}  {note}')
responsive_2 = [p for p in range(n) if d2[p] == num_terms_2]
results_2 = show_query_results('AND: Aspen AND Sassafras', ['Aspen','Sassafr
                               responsive_2, pi_tilde, labels, page_titles,
```

```
-- Query 2: AND Aspen AND Sassafras (requires d == 2) --
  Page A: d = 0
  Page B: d = 1  <-- only 1/2 terms
  Page C: d = 0
  Page D: d = 2  <-- BOTH present (responsive!)
  Page E: d = 0
  Page F: d = 2  <-- BOTH present (responsive!)
  Page G: d = 0
  Page H: d = 1  <-- only 1/2 terms
  Page I: d = 1  <-- only 1/2 terms
  Page J: d = 2  <-- BOTH present (responsive!)
  Page K: d = 1  <-- only 1/2 terms
  Page L: d = 2  <-- BOTH present (responsive!)

  ============================================================
  QUERY: AND: Aspen AND Sassafras
  ============================================================
  Result   Page   Title                      PageRank     Matching Keywords
  ---------------------------------------------------------------------------
-
  #1      J      The Warped Board          0.129794   Aspen, Sassafras
  #2      F      Good Wood                 0.124250   Aspen, Sassafras
  #3      D      Shady Place               0.088495   Aspen, Sassafras
  #4      L      Sappy Planks              0.081375   Aspen, Sassafras
```

In [21]:

```python
# Query 3: OR -- 'Willow OR Redwood'
q3 = np.zeros(len(keywords))
q3[kw['Willow']] = 1; q3[kw['Redwood']] = 1
d3 = T.T @ q3
print('-- Query 3: OR Willow OR Redwood (requires d >= 1) --')
for p in range(n):
    present = []
    if T[kw['Willow'], p] == 1:  present.append('Willow')
    if T[kw['Redwood'], p] == 1: present.append('Redwood')
    note = f'<-- has: {present}' if present else ''
    print(f'  Page {labels[p]}: d = {d3[p]:.0f}  {note}')
responsive_3 = [p for p in range(n) if d3[p] >= 1]
results_3 = show_query_results('OR: Willow OR Redwood', ['Willow','Redwood']
                               responsive_3, pi_tilde, labels, page_titles,
```

```
-- Query 3: OR Willow OR Redwood (requires d >= 1) --
  Page A: d = 0
  Page B: d = 2  <-- has: ['Willow', 'Redwood']
  Page C: d = 1  <-- has: ['Redwood']
  Page D: d = 0
  Page E: d = 2  <-- has: ['Willow', 'Redwood']
  Page F: d = 1  <-- has: ['Redwood']
  Page G: d = 1  <-- has: ['Redwood']
  Page H: d = 2  <-- has: ['Willow', 'Redwood']
  Page I: d = 0
  Page J: d = 2  <-- has: ['Willow', 'Redwood']
  Page K: d = 1  <-- has: ['Redwood']
  Page L: d = 0

  ============================================================
  QUERY: OR: Willow OR Redwood
  ============================================================
  Result   Page   Title                      PageRank     Matching Keywords
  ---------------------------------------------------------------------------
-
  #1      G      Lumber Jack Man           0.145502   Redwood
  #2      J      The Warped Board          0.129794   Willow, Redwood
  #3      F      Good Wood                 0.124250   Redwood
  #4      H      Timber!                   0.081030   Willow, Redwood
  #5      K      Evergreen Plants          0.078004   Redwood
  #6      B      The Weeping Willow        0.057344   Willow, Redwood
  #7      E      The Prickly Pine          0.028287   Willow, Redwood
  #8      C      Leaf Fall                 0.024686   Redwood
```

In [22]:

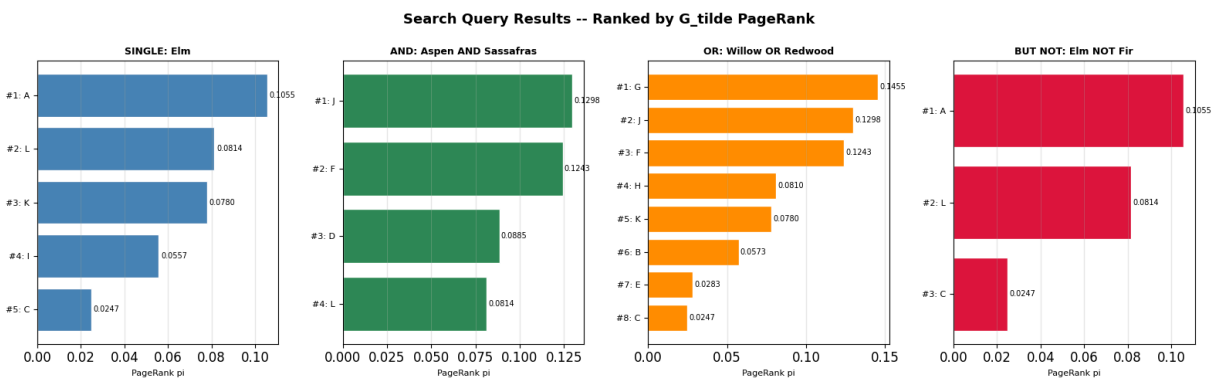
```python
# Query 4: BUT NOT -- 'Elm BUT NOT Fir'
has_elm = set(p for p in range(n) if T[kw['Elm'], p] == 1)
has_fir = set(p for p in range(n) if T[kw['Fir'], p] == 1)
print(f'Pages with Elm:          {sorted([labels[p] for p in has_elm])}')
print(f'Pages with Fir:          {sorted([labels[p] for p in has_fir])}')
print(f'Pages with Elm NOT Fir:  {sorted([labels[p] for p in has_elm - has_f
responsive_4 = sorted(has_elm - has_fir)
results_4 = show_query_results('BUT NOT: Elm BUT NOT Fir', ['Elm'],
                               responsive_4, pi_tilde, labels, page_titles,
```

```
Pages with Elm:          ['A', 'C', 'I', 'K', 'L']
Pages with Fir:          ['B', 'D', 'F', 'I', 'K']
Pages with Elm NOT Fir:  ['A', 'C', 'L']

  ============================================================
  QUERY: BUT NOT: Elm BUT NOT Fir
  ============================================================
  Result   Page   Title                      PageRank     Matching Keywords
  ---------------------------------------------------------------------------
-
  #1      A      Trees For All             0.105515   Elm
  #2      L      Sappy Planks              0.081375   Elm
  #3      C      Leaf Fall                 0.024686   Elm
```

In [23]:

```python
# Four-panel summary chart (Slides 19-22)
queries_chart = [
    ('SINGLE: Elm',              ['Elm'],              results_1),
```

```
    ('AND: Aspen AND Sassafras', ['Aspen','Sassafras'], results_2),
    ('OR: Willow OR Redwood',    ['Willow','Redwood'],  results_3),
    ('BUT NOT: Elm NOT Fir',     ['Elm'],               results_4),
]
fig, axes = plt.subplots(1, 4, figsize=(16, 5))
colors_chart = ['steelblue', 'seagreen', 'darkorange', 'crimson']
for ax, (name, qkws, results), color in zip(axes, queries_chart, colors_char
    if not results:
        ax.text(0.5, 0.5, 'No results', ha='center', va='center', transform=
        ax.set_title(name, fontsize=9, fontweight='bold'); continue
    scores = [pi_tilde[i] for i in results]
    bars   = ax.barh(range(len(results)), scores, color=color, edgecolor='wh
    ax.set_yticks(range(len(results)))
    ax.set_yticklabels([f'#{r+1}: {labels[i]}' for r, i in enumerate(results
    ax.invert_yaxis()
    ax.set_xlabel('PageRank pi', fontsize=8)
    ax.set_title(name, fontsize=9, fontweight='bold')
    ax.grid(axis='x', alpha=0.3)
    for bar, val in zip(bars, scores):
        ax.text(val + 0.001, bar.get_y() + bar.get_height()/2, f'{val:.4f}',
plt.suptitle('Search Query Results -- Ranked by G_tilde PageRank', fontsize=
plt.tight_layout()
plt.savefig('all_query_results.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved: all_query_results.png')
```




```
Saved: all_query_results.png
```

# Section 13 — Complete Results Summary

In [24]:

```
print('=' * 70)
print('  MAT 167 FINAL PROJECT -- COMPLETE RESULTS SUMMARY')
print('  Google Network #10  |  Keywords: Trees Set #3')
print('=' * 70)

print('\n--- PageRank Rankings ---')
print(f"  {'Rank':<6} {'G_fixed page':<22} {'pi(G_fixed)':<14} {'G_tilde pag
print('  ' + '-' * 75)
for r in range(n):
    i = rank_fixed_arr[r]; j = rank_tilde_arr[r]
    print(f'  #{r+1:<4} {labels[i]+": "+page_titles[labels[i]]:<22} {pi_fixe
          f' {labels[j]+": "+page_titles[labels[j]]:<22} {pi_tilde[j]:.6f}')

print(f'\n--- Power Method ---')
```

```
print(f'  G_fixed:  converged in {iters_fixed} iterations')
print(f'  G_tilde:  converged in {iters_tilde} iterations  (alpha={alpha} bc
print(f'  G_fixed matches eigendecomp: {np.max(np.abs(pi_power_fixed - pi_fi
print(f'  G_tilde matches eigendecomp: {np.max(np.abs(pi_power_tilde - pi_ti

print('\n--- Search Query Results (ranked by G_tilde PageRank) ---')
for name, qkws, results in [
    ('SINGLE: Elm',              ['Elm'],               results_1),
    ('AND: Aspen AND Sassafras', ['Aspen','Sassafras'],  results_2),
    ('OR: Willow OR Redwood',    ['Willow','Redwood'],   results_3),
    ('BUT NOT: Elm BUT NOT Fir', ['Elm'],               results_4),
]:
    print(f'\n  {name}:')
    for rank, i in enumerate(results, 1):
        matching = [k for k in qkws if k in page_keywords.get(labels[i], [])
        matching_str = ', '.join(matching)
        print(f'    #{rank}: {labels[i]} -- {page_titles[labels[i]]}  (pi =

print('\n' + '=' * 70)
print('  All computations complete.')
print('=' * 70)
```

```
======================================================================
  MAT 167 FINAL PROJECT -- COMPLETE RESULTS SUMMARY
  Google Network #10  |  Keywords: Trees Set #3
======================================================================

--- PageRank Rankings ---
  Rank   G_fixed page           pi(G_fixed)    G_tilde page           pi(G_t
ilde)
  ---------------------------------------------------------------------------
-
  #1    G: Lumber Jack Man     0.159855       G: Lumber Jack Man     0.14550
2
  #2    J: The Warped Board    0.140175       J: The Warped Board    0.12979
4
  #3    F: Good Wood           0.126678       F: Good Wood           0.12425
0
  #4    A: Trees For All       0.104962       A: Trees For All       0.10551
5
  #5    D: Shady Place         0.088674       D: Shady Place         0.08849
5
  #6    H: Timber!             0.084301       L: Sappy Planks        0.08137
5
  #7    L: Sappy Planks        0.084301       H: Timber!             0.08103
0
  #8    K: Evergreen Plants    0.082114       K: Evergreen Plants    0.07800
4
  #9    B: The Weeping Willow  0.052481       B: The Weeping Willow  0.05734
4
  #10   I: Fine Grains         0.047504       I: Fine Grains         0.05571
9
  #11   E: The Prickly Pine    0.015835       E: The Prickly Pine    0.02828
7
  #12   C: Leaf Fall           0.013120       C: Leaf Fall           0.02468
6

--- Power Method ---
  G_fixed:  converged in 89 iterations
  G_tilde:  converged in 54 iterations  (alpha=0.85 bounds rate)
  G_fixed matches eigendecomp: True
  G_tilde matches eigendecomp: True

--- Search Query Results (ranked by G_tilde PageRank) ---

  SINGLE: Elm:
    #1: A -- Trees For All  (pi = 0.105515)  [Elm]
    #2: L -- Sappy Planks  (pi = 0.081375)  [Elm]
    #3: K -- Evergreen Plants  (pi = 0.078004)  [Elm]
    #4: I -- Fine Grains  (pi = 0.055719)  [Elm]
    #5: C -- Leaf Fall  (pi = 0.024686)  [Elm]

  AND: Aspen AND Sassafras:
    #1: J -- The Warped Board  (pi = 0.129794)  [Aspen, Sassafras]
    #2: F -- Good Wood  (pi = 0.124250)  [Aspen, Sassafras]
    #3: D -- Shady Place  (pi = 0.088495)  [Aspen, Sassafras]
    #4: L -- Sappy Planks  (pi = 0.081375)  [Aspen, Sassafras]
```

```
  OR: Willow OR Redwood:
    #1: G -- Lumber Jack Man  (pi = 0.145502)  [Redwood]
    #2: J -- The Warped Board  (pi = 0.129794)  [Willow, Redwood]
    #3: F -- Good Wood  (pi = 0.124250)  [Redwood]
    #4: H -- Timber!  (pi = 0.081030)  [Willow, Redwood]
    #5: K -- Evergreen Plants  (pi = 0.078004)  [Redwood]
    #6: B -- The Weeping Willow  (pi = 0.057344)  [Willow, Redwood]
    #7: E -- The Prickly Pine  (pi = 0.028287)  [Willow, Redwood]
    #8: C -- Leaf Fall  (pi = 0.024686)  [Redwood]

  BUT NOT: Elm BUT NOT Fir:
    #1: A -- Trees For All  (pi = 0.105515)  [Elm]
    #2: L -- Sappy Planks  (pi = 0.081375)  [Elm]
    #3: C -- Leaf Fall  (pi = 0.024686)  [Elm]

======================================================================
  All computations complete.
======================================================================
```